

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	RPC Vs. gRPC				
NÚMERO DE LA PRÁCTICA:	05	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	19/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/I5 SOLUCIÓN DE EJERCICIO RESUELTO Ejercicio Resuelto: Calculadora con gRPC El archivo calculator.proto define la estructura del servicio de calculadora distribuida utilizando Protocol Buffers, especificando el método Sum y los mensajes de solicitud y respuesta. <pre> service CalculatorService { rpc Sum (SumRequest) returns (SumResponse); } message SumRequest { int32 a = 1; int32 b = 2; } message SumResponse { int32 result = 1; } </pre> La implementación del servicio en CalculatorService.java extiende la base generada por gRPC para realizar la operación aritmética de suma sobre los parámetros recibidos. <pre> @Override public void sum(SumRequest req, StreamObserver<SumResponse> responseObserver) { int result = req.getA() + req.getB(); SumResponse response = SumResponse.newBuilder().setResult(result).build(); responseObserver.onNext(response); responseObserver.onCompleted(); } </pre> A continuación se muestra la ejecución del cliente gRPC, donde se realiza una petición de suma al servidor y se recibe el resultado de forma exitosa.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
> java -cp "target/classes:target/dependency/*" com.lab05.er1.Client
Resultado: 12
>
```

Figura 1: Ejecución del cliente de la calculadora gRPC.

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Ejercicio 1: Servicio RPC Tradicional (RMI)

Se implementó una calculadora utilizando Java RMI para demostrar el enfoque tradicional de RPC, donde la interfaz remota define las operaciones de multiplicación, división y potencia.

```
double multiply(double a, double b) throws RemoteException;
double divide(double a, double b) throws RemoteException;
double add(double a, double b) throws RemoteException;
double subtract(double a, double b) throws RemoteException;
double power(double a, double b) throws RemoteException;
```

La clase `Calculator.java` implementa la interfaz remota, definiendo el comportamiento de las operaciones aritméticas que serán invocadas por los clientes.

```
@Override
public double multiply(double a, double b) throws RemoteException {
    return a * b;
}

@Override
public double divide(double a, double b) throws RemoteException {
    return a / b;
}

@Override
public double power(double a, double b) throws RemoteException {
    return Math.pow(a, b);
}
```

El servidor registra el objeto remoto en el RMI Registry bajo un nombre específico, permitiendo que los clientes lo localicen e invoquen sus métodos de manera transparente.

```
Calculator calculator = new Calculator();

Registry registry = LocateRegistry.createRegistry(1099);

registry.rebind("CalculatorService", calculator);

System.out.println("Servidor RPC iniciado");
```

Para el cliente, se implementó una lógica de búsqueda en el registro para obtener la referencia del objeto remoto y proceder con las llamadas a los métodos.

```
Registry registry = LocateRegistry.getRegistry("localhost", 1099);
ICalculator calculator = (ICalculator) registry.lookup(
    "CalculatorService"
);
double result = calculator.add(5, 3);
System.out.println("Sum Result: " + result);
result = calculator.power(2, 10);
System.out.println("Power Result: " + result);
```

Para mejorar la usabilidad de la aplicación, se desarrolló una interfaz gráfica sencilla que permite interactuar con las operaciones de la calculadora de forma intuitiva.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

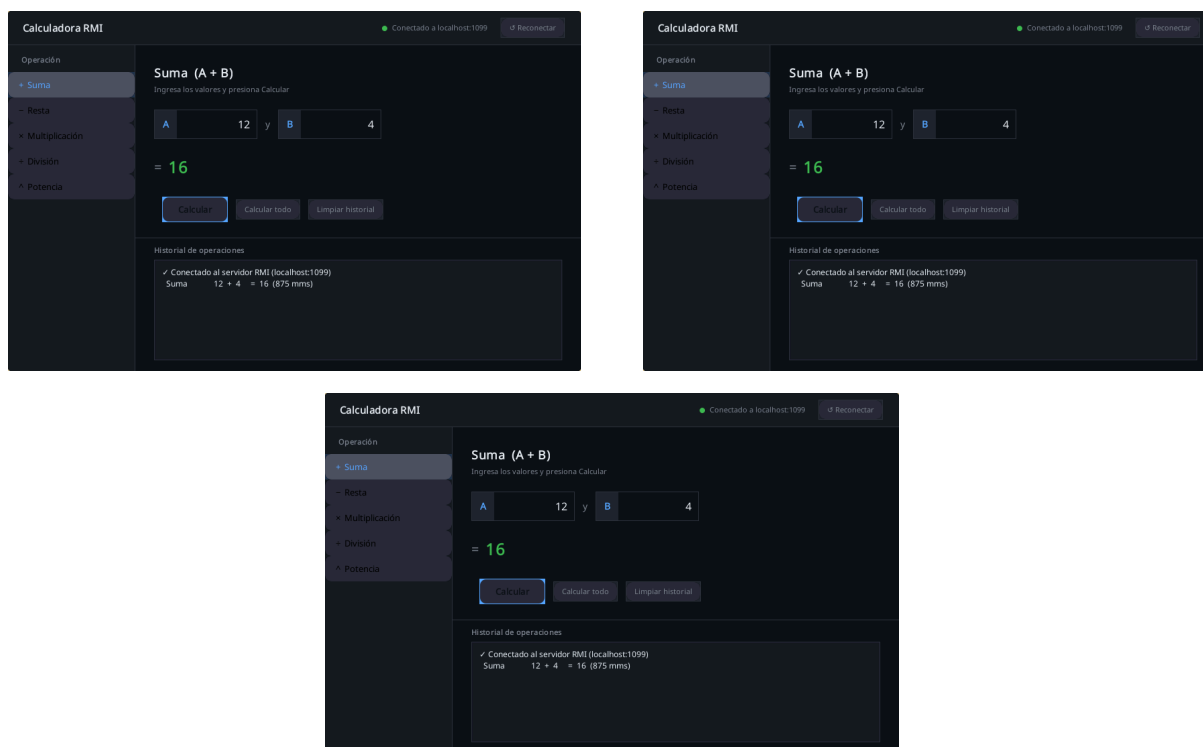


Figura 2: Interfaz gráfica de la calculadora RMI.

A continuación se presenta la ejecución del cliente de consola, validando la funcionalidad de las operaciones aritméticas distribuidas mediante RMI.

```
> java -cp "target/classes:target/dependency/*" com.lab05.ep1.CalculatorClient
Sum Result: 8.0
Product Result: 15.0
Division Result: 1.6666666666666667
Subtraction Result: 2.0
Power Result: 125.0
>
```

Figura 3: Ejecución del cliente de consola de la calculadora RMI.

Ejercicio 2: Sistema de Conversión con gRPC

Este ejercicio consistió en implementar un servicio de conversión de unidades (temperatura, moneda, longitud, peso y tiempo) utilizando gRPC para aprovechar su eficiencia binaria.

```
service ConverterService {
    rpc ConvertCelsiusToFahrenheit(ConvertRequest) returns (ConvertResponse);
    rpc ConvertFahrenheitToCelsius(ConvertRequest) returns (ConvertResponse);
    rpc ConvertSolesToDollars(ConvertRequest) returns (ConvertResponse);
    rpc ConvertDollarsToSoles(ConvertRequest) returns (ConvertResponse);
    rpc ConvertKmToMiles(ConvertRequest) returns (ConvertResponse);
    rpc ConvertMilesToKm(ConvertRequest) returns (ConvertResponse);
    rpc ConvertKgToLbs(ConvertRequest) returns (ConvertResponse);
    rpc ConvertLbsToKg(ConvertRequest) returns (ConvertResponse);
    rpc ConvertHoursToMinutes(ConvertRequest) returns (ConvertResponse);
    rpc ConvertMinutesToHours(ConvertRequest) returns (ConvertResponse);
}

message ConvertRequest {
    double input = 1;
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

}

message ConvertResponse {
    double result = 1;
}

```

El servicio ConverterService.java implementa la lógica de negocio para cada tipo de conversión, procesando los valores de entrada y retornando el resultado calculado.

```

@Override
public void convertCelsiusToFahrenheit(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 1.8 + 32;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertSolesToDollars(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 0.23;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertKmToMiles(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 0.621371;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

```

El servidor se encarga de exponer el servicio en un puerto específico utilizando ServerBuilder de gRPC, gestionando el ciclo de vida del proceso.

```

Server server = ServerBuilder.forPort(50051)
    .addService(new ConverterService())
    .build();

server.start();
server.awaitTermination();

```

El cliente de consola utiliza un canal de comunicación para enviar peticiones al servidor y mostrar los resultados de las conversiones de forma interactiva.

```

ManagedChannel channel = ManagedChannelBuilder.forAddress(
    "localhost",
    50051
)
    .usePlaintext()
    .build();
ConverterServiceGrpc.ConverterServiceBlockingStub stub =
    ConverterServiceGrpc.newBlockingStub(channel);

ConvertRequest request = ConvertRequest.newBuilder().setInput(12).build();

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```
ConvertResponse response = stub.convertCelsiusToFahrenheit(request);
System.out.println("Resultado: " + response.getResult());
```

Se diseñó una interfaz gráfica para facilitar el acceso a las múltiples conversiones disponibles, proporcionando una experiencia de usuario más amigable.

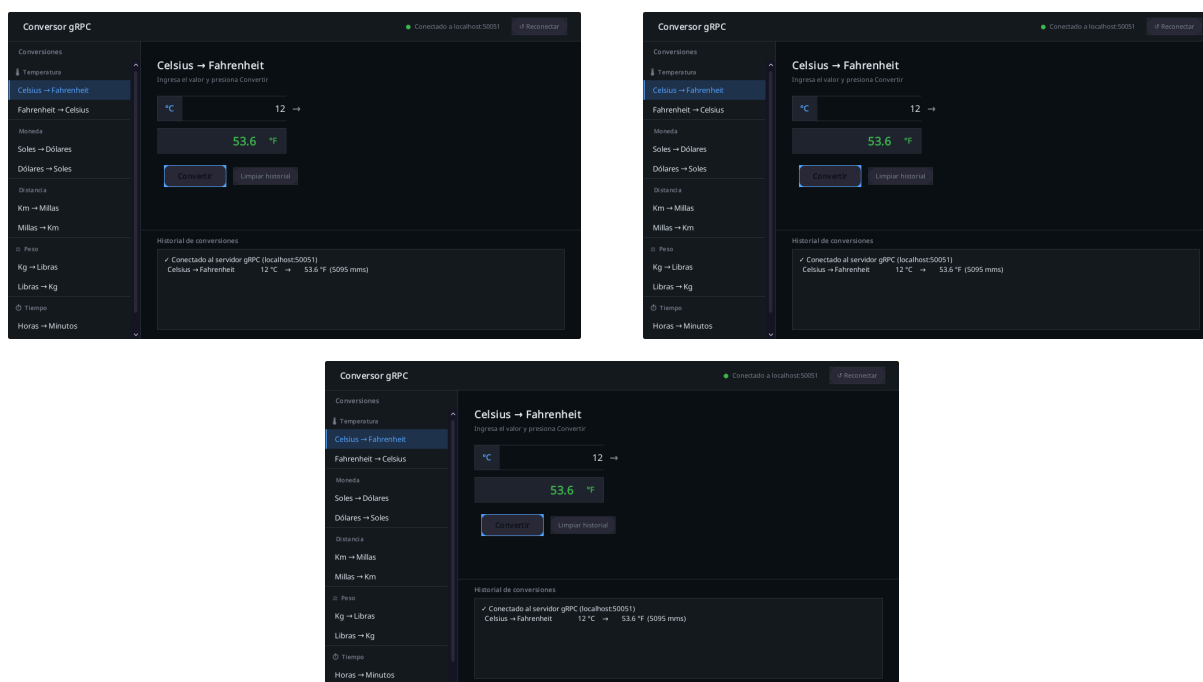


Figura 4: Interfaz gráfica del sistema de conversión gRPC.

A continuación se observa la interacción con el cliente de consola, donde se ejecutan diversas pruebas de conversión seleccionando opciones desde un menú interactivo.

```

> java -cp "target/classes:target/dependency/*" com.lab05.ep2.Client
Menu de conversión:
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Soles a Dólares
4. Dólares a Soles
5. Km a Millas
6. Millas a Km
7. Kg a Lbs
8. Lbs a Kg
9. Horas a Minutos
10. Minutos a Horas
0. Salir
Ingrese una opción:
1
Ingrese el valor a convertir:
25
Resultado: 77.0
Menu de conversión:
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Soles a Dólares
4. Dólares a Soles
5. Km a Millas
6. Millas a Km
7. Kg a Lbs
8. Lbs a Kg
9. Horas a Minutos
10. Minutos a Horas
0. Salir
Ingrese una opción:
3
Ingrese el valor a convertir:
100
Resultado: 23.0
Menu de conversión:
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Soles a Dólares
4. Dólares a Soles
5. Km a Millas
6. Millas a Km
7. Kg a Lbs
8. Lbs a Kg
9. Horas a Minutos
10. Minutos a Horas
0. Salir
Ingrese una opción:
5
Ingrese el valor a convertir:
10
Resultado: 6.21371
Menu de conversión:
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Soles a Dólares
4. Dólares a Soles
5. Km a Millas
6. Millas a Km
7. Kg a Lbs
8. Lbs a Kg
9. Horas a Minutos
10. Minutos a Horas
0. Salir
Ingrese una opción:
0
Saliendo...
>

```

Figura 5: Ejecución del cliente de consola del sistema de conversión gRPC.

ACTIVIDAD COMPARATIVA

Para evaluar el rendimiento de ambos enfoques, se realizaron pruebas de carga consistentes en ráfagas de peticiones. A continuación se presentan las capturas del consumo de memoria de cada servidor durante el procesamiento de solicitudes:



Figura 6: Comparativa del consumo de memoria entre RMI (izquierda) y gRPC (derecha).

A continuación se detallan los tiempos de respuesta obtenidos durante las pruebas de latencia para cada tecnología, los cuales sirvieron de base para el cálculo del promedio presentado en la tabla comparativa:

Celsius → Fahrenheit	12 °C	→	53.6 °F	(1670 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1434 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1620 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1451 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1496 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1436 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1427 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1503 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1437 mms)
Celsius → Fahrenheit	12 °C	→	53.6 °F	(1488 mms)

gRPC (Conversión)

Suma	12	+	12	=	24	(513 mms)
Suma	12	+	12	=	24	(571 mms)
Suma	12	+	12	=	24	(698 mms)
Suma	12	+	12	=	24	(757 mms)
Suma	12	+	12	=	24	(581 mms)
Suma	12	+	12	=	24	(540 mms)
Suma	12	+	12	=	24	(530 mms)
Suma	12	+	12	=	24	(548 mms)
Suma	12	+	12	=	24	(394 mms)
Suma	12	+	12	=	24	(551 mms)

RMI (Suma)

A partir de las pruebas realizadas, se presenta la siguiente tabla comparativa entre ambos enfoques:

Métrica	RPC Tradicional (RMI)	gRPC
Tiempo respuesta (Promedio)	568ms	1496ms
Complejidad	Media	Alta
Escalabilidad	Limitada (Ecosistema Java)	Alta (Multiplataforma)

Se observa que en las pruebas locales, RMI presenta un tiempo de respuesta inferior al de gRPC. Esto se explica porque RMI, al operar de forma nativa dentro del ecosistema Java y en un entorno local, no sufre el overhead de traducción de protocolos ni latencia de red significativa.

Sin embargo, en un entorno de producción real con retardos de red considerables, gRPC escalaría de manera más eficiente. Su uso de HTTP/2 para multiplexación y la serialización binaria compacta con Protocol Buffers compensarían el overhead de traducción, superando a RMI en velocidad de respuesta y gestión de recursos ante un gran volumen de tráfico distribuido.

CUESTIONARIO

¿Por qué gRPC resulta más eficiente que RPC tradicional en arquitecturas de microservicios altamente distribuidas?

gRPC utiliza HTTP/2 para el transporte, lo que permite la multiplexación de múltiples peticiones sobre una sola conexión TCP, reduciendo significativamente la latencia. Además, emplea Protocol Buffers para la serialización binaria, lo que genera mensajes mucho más pequeños y rápidos de procesar que los formatos de texto como XML o JSON.

¿Qué limitaciones podría presentar gRPC en entornos donde la interoperabilidad humana (depuración manual o pruebas directas) es necesaria?

Al ser un protocolo binario, no es posible leer o modificar los mensajes directamente sin herramientas especializadas como gRPCurl o Postman. Esto complica la depuración rápida mediante el uso de comandos simples de red o la inspección directa de los flujos de datos en el navegador o consola.

Si diseñaras una plataforma bancaria distribuida, ¿qué factores arquitectónicos te harían elegir RPC tradicional o gRPC?

Elegiría gRPC principalmente por su soporte nativo para contratos estrictos mediante archivos proto, lo que garantiza la integridad de los datos financieros. También pesaría la necesidad de comunicación bidireccional y streaming para actualizaciones de saldos en tiempo real, junto con la eficiencia en entornos de alta concurrencia.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. gRPC demuestra ser notablemente más eficiente que RMI en términos de latencia y tamaño de carga útil debido a su serialización binaria compacta.
2. La arquitectura de gRPC facilita enormemente la interoperabilidad entre diferentes lenguajes de programación, superando la limitación de RMI que depende del ecosistema Java.
3. El uso de HTTP/2 en gRPC permite una gestión de conexiones más robusta y moderna, optimizando el rendimiento en redes con alta latencia o gran volumen de tráfico.

RECOMENDACIONES

1. Se recomienda migrar sistemas antiguos basados en RPC tradicional hacia gRPC cuando se requiera escalar horizontalmente o integrar microservicios políglotas.
2. Utilizar herramientas de reflexión de gRPC durante la etapa de desarrollo para mitigar la dificultad de depuración que conlleva el uso de protocolos binarios.
3. Definir cuidadosamente los archivos .proto para asegurar que los contratos de servicio sean claros y mantengan la compatibilidad hacia atrás durante la evolución del sistema.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- [4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
- [5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley.
- [6] gRPC Authors. (2026). gRPC Documentation. Recuperado de <https://grpc.io/docs/>

ANEXOS

Ejercicio 1: Servicio RPC Tradicional (RMI)

Interfaz Remota

```
package com.lab05.epl;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ICalculator extends Remote {
    double multiply(double a, double b) throws RemoteException;
    double divide(double a, double b) throws RemoteException;
    double add(double a, double b) throws RemoteException;
    double subtract(double a, double b) throws RemoteException;
    double power(double a, double b) throws RemoteException;
}
```

Implementación

```
package com.lab05.epl;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class Calculator extends UnicastRemoteObject implements ICalculator {

    public Calculator() throws RemoteException {
        super();
    }

    @Override
    public double multiply(double a, double b) throws RemoteException {
        return a * b;
    }

    @Override
    public double divide(double a, double b) throws RemoteException {
        return a / b;
    }

    @Override
    public double add(double a, double b) throws RemoteException {
        return a + b;
    }

    @Override
    public double subtract(double a, double b) throws RemoteException {
        return a - b;
    }

    @Override
    public double power(double a, double b) throws RemoteException {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```

    return Math.pow(a, b);
}
}

```

Servidor

```

package com.lab05.ep1;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {

    public static void main(String[] args) {
        try {
            Calculator calculator = new Calculator();

            Registry registry = LocateRegistry.createRegistry(1099);

            registry.rebind("CalculatorService", calculator);

            System.out.println("Servidor RPC iniciado");
            while (true) {}
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Cliente CLI

```

package com.lab05.ep1;

import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {

    public static void main(String[] args) {
        try {
            double a = 5;
            double b = 3;
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            ICalculator calculator = (ICalculator) registry.lookup(
                "CalculatorService"
            );
            double result = calculator.add(a, b);
            System.out.println("Sum Result: " + result);
            result = calculator.multiply(a, b);
            System.out.println("Product Result: " + result);
            result = calculator.divide(a, b);
            System.out.println("Division Result: " + result);
            result = calculator.subtract(a, b);
            System.out.println("Subtraction Result: " + result);
            result = calculator.power(a, b);
            System.out.println("Power Result: " + result);
        } catch (RemoteException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

Cliente GUI

```
package com.lab05.ep1;

import java.awt.*;
import java.awt.event.*;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.text.DecimalFormat;
import javax.swing.*;
import javax.swing.border.*;

public class CalculatorClientGUI extends JFrame {

    // --- RMI ---
    private ICalculator calculator;
    private boolean connected = false;

    // --- Colores ---
    private static final Color BG_DARK = new Color(13, 17, 23);
    private static final Color BG_CARD = new Color(22, 27, 34);
    private static final Color BG_INPUT = new Color(33, 38, 45);
    private static final Color ACCENT = new Color(88, 166, 255);
    private static final Color ACCENT_H = new Color(121, 192, 255);
    private static final Color SUCCESS = new Color(63, 185, 80);
    private static final Color WARNING = new Color(210, 153, 34);
    private static final Color DANGER = new Color(248, 81, 73);
    private static final Color TEXT_PRI = new Color(230, 237, 243);
    private static final Color TEXT_MUT = new Color(125, 133, 144);
    private static final Color BORDER_C = new Color(48, 54, 61);

    // --- Widgets ---
    private JTextField fieldA, fieldB;
    private JLabel resultLabel, opLabel;
    private JLabel statusDot, statusText;
    private JTextArea historyArea;

    // Operaciones: nombre, símbolo
    private static final String[][] OPS = {
        {"Suma", "+"},
        {"Resta", "-"},
        {"Multiplicación", "x"},
        {"División", "÷"},
        {"Potencia", "^"},
    };

    private int selectedOp = 0; // índice en OPS

    public CalculatorClientGUI() {
        setTitle("Calculadora RMI - Lab 05");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(680, 560);
        setMinimumSize(new Dimension(580, 480));
        setLocationRelativeTo(null);
        buildUI();
        connectRMI();
    }

    // ---
    private void buildUI() {
        JPanel root = new JPanel(new BorderLayout());
        root.setBackground(BG_DARK);
        root.add(buildTopBar(), BorderLayout.NORTH);
        root.add(buildLeftPanel(), BorderLayout.WEST);
        root.add(buildCenter(), BorderLayout.CENTER);
        setContentPane(root);
    }

    // --- Barra superior ---
    private JPanel buildTopBar() {
        JPanel bar = new JPanel(new BorderLayout());
        bar.setBackground(BG_CARD);
        bar.setBorder(
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

        new CompoundBorder(
            new MatteBorder(0, 0, 1, 0, BORDER_C),
            new EmptyBorder(10, 18, 10, 18)
        )
    );

    JLabel title = new JLabel(" Calculadora RMI");
    title.setFont(new Font("Segoe UI", Font.BOLD, 15));
    title.setForeground(TEXT_PRI);

    JPanel statusPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 6, 0));
    statusPanel.setOpaque(false);

    statusDot = new JLabel("●");
    statusDot.setFont(new Font("Dialog", Font.PLAIN, 10));
    statusDot.setForeground(WARNING);

    statusText = new JLabel("Conectando...");
    statusText.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    statusText.setForeground(TEXT_MUT);

    JButton reconnBtn = smallBtn("Reconectar");
    reconnBtn.addActionListener(e -> connectRMI());

    statusPanel.add(statusDot);
    statusPanel.add(statusText);
    statusPanel.add(Box.createHorizontalStrut(8));
    statusPanel.add(reconnBtn);

    bar.add(title, BorderLayout.WEST);
    bar.add(statusPanel, BorderLayout.EAST);
    return bar;
}

// — Panel izquierdo: botones de operación —
private JPanel buildLeftPanel() {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setPreferredSize(new Dimension(190, 0));
    panel.setBackground(BG_CARD);
    panel.setBorder(
        new CompoundBorder(
            new MatteBorder(0, 0, 0, 1, BORDER_C),
            new EmptyBorder(14, 0, 14, 0)
        )
    );

    JLabel header = new JLabel(" Operación");
    header.setFont(new Font("Segoe UI", Font.BOLD, 11));
    header.setForeground(TEXT_MUT);
    header.setBorder(new EmptyBorder(0, 14, 10, 0));
    header.setAlignmentX(Component.LEFT_ALIGNMENT);
    panel.add(header);

    ButtonGroup group = new ButtonGroup();
    for (int i = 0; i < OPS.length; i++) {
        final int idx = i;
        JToggleButton btn = new JToggleButton(OPS[i][1] + " " + OPS[i][0]);
        btn.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        btn.setHorizontalAlignment(SwingConstants.LEFT);
        btn.setBackground(BG_CARD);
        btn.setBorder(new EmptyBorder(10, 16, 10, 16));
        btn.setFocusPainted(false);
        btn.setOpaque(true);
        btn.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
        btn.setAlignmentX(Component.LEFT_ALIGNMENT);
        btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

        if (i == 0) btn.setSelected(true);

        btn.addMouseListener(
            new MouseAdapter() {

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

        @Override
        public void mouseEntered(MouseEvent e) {
            if (!btn.isSelected()) btn.setBackground(BG_INPUT);
        }

        @Override
        public void mouseExited(MouseEvent e) {
            if (!btn.isSelected()) btn.setBackground(BG_CARD);
        }
    }
};

btn.addActionListener(e -> {
    selectedOp = idx;
    updateOpLabel();
    // resetear color de los demás
    for (Component c : panel.getComponents()) {
        if (c instanceof JToggleButton tb && tb != btn) {
            tb.setBackground(BG_CARD);
            tb.setForeground(new Color(0, 0, 0));
        }
    }
    btn.setBackground(new Color(33, 52, 78));
    btn.setForeground(ACCENT);
});

group.add(btn);
panel.add(btn);
}

panel.add(Box.createVerticalGlue());
return panel;
}

// — Panel central —
private JPanel buildCenter() {
    JPanel center = new JPanel(new BorderLayout());
    center.setBackground(BG_DARK);

    // — Área de cálculo —
    JPanel calcPanel = new JPanel();
    calcPanel.setLayout(new BoxLayout(calcPanel, BoxLayout.Y_AXIS));
    calcPanel.setBackground(BG_DARK);
    calcPanel.setBorder(new EmptyBorder(28, 28, 20, 28));

    opLabel = new JLabel("Suma (A + B)");
    opLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
    opLabel.setForeground(TEXT_PRI);
    opLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

    JLabel sub = new JLabel("Ingresa los valores y presiona Calcular");
    sub.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    sub.setForeground(TEXT_MUT);
    sub.setAlignmentX(Component.LEFT_ALIGNMENT);
    sub.setBorder(new EmptyBorder(4, 0, 22, 0));

    // Fila de entradas
    JPanel inputRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
    inputRow.setOpaque(false);
    inputRow.setAlignmentX(Component.LEFT_ALIGNMENT);

    fieldA = styledField();
    fieldB = styledField();

    JLabel lblA = inputTag("A");
    JLabel lblB = inputTag("B");
    JLabel sep = new JLabel(" y ");
    sep.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    sep.setForeground(TEXT_MUT);

    inputRow.add(lblA);
    inputRow.add(fieldA);

```



```
inputRow.add(sep);
inputRow.add(lblB);
inputRow.add(fieldB);

// Resultado
JPanel resultRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
resultRow.setOpaque(false);
resultRow.setAlignmentX(Component.LEFT_ALIGNMENT);

JLabel eqLabel = new JLabel("=");
eqLabel.setFont(new Font("Segoe UI", Font.PLAIN, 20));
eqLabel.setForeground(TEXT_MUT);

resultLabel = new JLabel("-");
resultLabel.setFont(new Font("JetBrains Mono", Font.BOLD, 24));
resultLabel.setForeground(SUCCESS);
resultLabel.setPreferredSize(new Dimension(260, 48));
resultLabel.setHorizontalAlignment(SwingConstants.LEFT);

resultRow.add(eqLabel);
resultRow.add(resultLabel);

// Botones
JPanel btnRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 0));
btnRow.setOpaque(false);
btnRow.setAlignmentX(Component.LEFT_ALIGNMENT);

JButton calcBtn = accentBtn("Calcular");
calcBtn.addActionListener(e -> doCalculate());

JButton allBtn = smallBtn("Calcular todo");
allBtn.addActionListener(e -> doCalculateAll());

JButton clearBtn = smallBtn("Limpiar historial");
clearBtn.addActionListener(e -> historyArea.setText(""));

btnRow.add(calcBtn);
btnRow.add(allBtn);
btnRow.add(clearBtn);

calcPanel.add(opLabel);
calcPanel.add(sub);
calcPanel.add(inputRow);
calcPanel.add(Box.createVerticalStrut(16));
calcPanel.add(resultRow);
calcPanel.add(Box.createVerticalStrut(20));
calcPanel.add(btnRow);

// — Historial —
JPanel histPanel = new JPanel(new BorderLayout());
histPanel.setBackground(BG_DARK);
histPanel.setBorder(
    new CompoundBorder(
        new MatteBorder(1, 0, 0, 0, BORDER_C),
        new EmptyBorder(0, 28, 16, 28)
    )
);

JLabel histHeader = new JLabel("Historial de operaciones");
histHeader.setFont(new Font("Segoe UI", Font.BOLD, 11));
histHeader.setForeground(TEXT_MUT);
histHeader.setBorder(new EmptyBorder(10, 0, 6, 0));

historyArea = new JTextArea();
historyArea.setEditable(false);
historyArea.setBackground(BG_CARD);
historyArea.setForeground(TEXT_PRI);
historyArea.setFont(new Font("JetBrains Mono", Font.PLAIN, 12));
historyArea.setBorder(new EmptyBorder(8, 10, 8, 10));

JScrollPane scroll = new JScrollPane(historyArea);
scroll.setBorder(new LineBorder(BORDER_C, 1));
```

```

scroll.getViewport().setBackground(BG_CARD);
scroll.setPreferredSize(new Dimension(0, 150));

histPanel.add(histHeader, BorderLayout.NORTH);
histPanel.add(scroll, BorderLayout.CENTER);

center.add(calcPanel, BorderLayout.CENTER);
center.add(histPanel, BorderLayout.SOUTH);
return center;
}

// -----
// Lógica RMI
// -----
private void connectRMI() {
    statusDot.setForeground(WARNING);
    statusText.setText("Conectando...");
    new SwingWorker<Boolean, Void>() {
        @Override
        protected Boolean doInBackground() {
            try {
                Registry registry = LocateRegistry.getRegistry("localhost", 1099);
                calculator = (ICalculator) registry.lookup("CalculatorService");
                calculator.add(0, 0); // prueba
                return true;
            } catch (Exception e) {
                return false;
            }
        }
    }.execute();

    @Override
    protected void done() {
        try {
            connected = get();
        } catch (Exception ex) {
            connected = false;
        }
        if (connected) {
            statusDot.setForeground(SUCCESS);
            statusText.setText("Conectado a localhost:1099");
            appendHistory("✓ Conectado al servidor RMI (localhost:1099)");
        } else {
            statusDot.setForeground(DANGER);
            statusText.setText("Sin conexión");
            appendHistory(
                "x No se pudo conectar. Verifica que el servidor esté activo."
            );
        }
    }
}

private void doCalculate() {
    if (!checkConnection()) return;
    double a, b;
    try {
        a = Double.parseDouble(fieldA.getText().trim().replace(",", "."));
        b = Double.parseDouble(fieldB.getText().trim().replace(",", "."));
    } catch (NumberFormatException ex) {
        resultLabel.setForeground(DANGER);
        resultLabel.setText("Valor inválido");
        return;
    }

    final int op = selectedOp;
    new SwingWorker<RCPRResult, Void>() {
        @Override
        protected RCPRResult doInBackground() throws Exception {
            return invoke(op, a, b);
        }
    }.execute();
}

```

```
protected void done() {
    try {
        RCPResult res = get();
        showResult(res.value());
        appendHistory(
            formatLine(OPS[op][0], OPS[op][1], a, b, res.value(), res.ms())
        );
    } catch (Exception ex) {
        resultLabel.setForeground(DANGER);
        resultLabel.setText("Error RMI");
        appendHistory("x Error: " + ex.getCause());
    }
}
}.execute();
}

/** Ejecuta las 5 operaciones en secuencia y las muestra en el historial. */
private void doCalculateAll() {
    if (!checkConnection()) return;
    double a, b;
    try {
        a = Double.parseDouble(fieldA.getText().trim().replace(",", "."));
        b = Double.parseDouble(fieldB.getText().trim().replace(",", "."));
    } catch (NumberFormatException ex) {
        resultLabel.setForeground(DANGER);
        resultLabel.setText("Valor inválido");
        return;
    }
    final double fa = a,
        fb = b;
    new SwingWorker<Void, Void>() {
        @Override
        protected Void doInBackground() throws Exception {
            appendHistory(
                "— Todas las operaciones (A=" + fa + ", B=" + fb + ") —"
            );
            for (int i = 0; i < OPS.length; i++) {
                try {
                    RCPResult res = invoke(i, fa, fb);
                    appendHistory(
                        formatLine(OPS[i][0], OPS[i][1], fa, fb, res.value(), res.ms())
                    );
                    if (i == selectedOp) SwingUtilities.invokeLater(() ->
                        showResult(res.value())
                    );
                } catch (Exception ex) {
                    appendHistory("x " + OPS[i][0] + ": " + ex.getMessage());
                }
            }
            return null;
        }
    }.execute();
}

private RCPResult invoke(int op, double a, double b) throws Exception {
    long start = System.nanoTime();
    double value = switch (op) {
        case 0 -> calculator.add(a, b);
        case 1 -> calculator.subtract(a, b);
        case 2 -> calculator.multiply(a, b);
        case 3 -> calculator.divide(a, b);
        case 4 -> calculator.power(a, b);
        default -> throw new IllegalStateException("Operación inválida");
    };
    long ms = (System.nanoTime() - start) / 1_000;
    return new RCPResult(value, ms);
}

// _____
// Helpers
// _____
private boolean checkConnection() {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```

if (!connected) {
    JOptionPane.showMessageDialog(
        this,
        "No hay conexión con el servidor RMI.\nPresiona o Reconectar.",
        "Sin conexión",
        JOptionPane.WARNING_MESSAGE
    );
}
return connected;
}

private void showResult(double res) {
    DecimalFormat df = new DecimalFormat("#,##0.#####");
    resultLabel.setForeground(SUCCESS);
    resultLabel.setText(df.format(res));
}

private String formatLine(
    String name,
    String sym,
    double a,
    double b,
    double res,
    long ms
) {
    DecimalFormat df = new DecimalFormat("#,##0.#####");
    return String.format(
        "%-16s %s %s %s %s = %s (%d mms)",
        name,
        df.format(a),
        sym,
        df.format(b),
        " ",
        df.format(res),
        ms
    );
}

private void appendHistory(String text) {
    SwingUtilities.invokeLater(() -> {
        historyArea.append(text + "\n");
        historyArea.setCaretPosition(historyArea.getDocument().getLength());
    });
}

private void updateOpLabel() {
    opLabel.setText(OPS[selectedOp][0] + " (A " + OPS[selectedOp][1] + " B)");
}

private JTextField styledField() {
    JTextField f = new JTextField("0");
    f.setPreferredSize(new Dimension(120, 44));
    f.setFont(new Font("JetBrains Mono", Font.PLAIN, 16));
    f.setBackground(BG_INPUT);
    f.setForeground(TEXT_PRI);
    f.setCaretColor(ACCENT);
    f.setBorder(
        new CompoundBorder(
            new MatteBorder(1, 0, 1, 1, BORDER_C),
            new EmptyBorder(0, 8, 0, 8)
        )
    );
    f.setHorizontalAlignment(JTextField.RIGHT);
    f.addActionListener(e -> doCalculate());
    return f;
}

private JLabel inputTag(String letter) {
    JLabel l = new JLabel(letter);
    l.setFont(new Font("Segoe UI", Font.BOLD, 14));
    l.setForeground(ACCENT);
    l.setPreferredSize(new Dimension(34, 44));
}

```

```
l.setHorizontalAlignment(SwingConstants.CENTER);
l.setBackground(BG_INPUT);
l.setOpaque(true);
l.setBorder(new MatteBorder(1, 1, 1, 0, BORDER_C));
return l;
}

private JButton accentBtn(String text) {
    JButton btn = new JButton(text);
    btn.setFont(new Font("Segoe UI", Font.BOLD, 13));
    btn.setBackground(ACCENT);
    btn.setForeground(BG_DARK);
    btn.setBorder(new EmptyBorder(10, 24, 10, 24));
    btn.setFocusPainted(false);
    btn.setOpaque(true);
    btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btn.addMouseListener(
        new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                btn.setBackground(ACCENT_H);
            }

            @Override
            public void mouseExited(MouseEvent e) {
                btn.setBackground(ACCENT);
            }
        }
    );
    return btn;
}

private JButton smallBtn(String text) {
    JButton btn = new JButton(text);
    btn.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    btn.setBackground(BG_INPUT);
    btn.setForeground(TEXT_MUT);
    btn.setBorder(
        new CompoundBorder(
            new LineBorder(BORDER_C, 1),
            new EmptyBorder(6, 12, 6, 12)
        )
    );
    btn.setFocusPainted(false);
    btn.setOpaque(true);
    btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btn.addMouseListener(
        new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                btn.setForeground(TEXT_PRI);
            }

            @Override
            public void mouseExited(MouseEvent e) {
                btn.setForeground(TEXT_MUT);
            }
        }
    );
    return btn;
}

// -----
public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception ignored) {}
    SwingUtilities.invokeLater(() ->
        new CalculatorClientGUI().setVisible(true)
    );
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

```
private record RCPResult(double value, long ms) {}
}
```

Ejercicio 2: Sistema de Conversión con gRPC

Definición Proto

```
syntax = "proto3";

package src.main.proto.ep2.v1;

service ConverterService {
  rpc ConvertCelsiusToFahrenheit(ConvertRequest) returns (ConvertResponse);
  rpc ConvertFahrenheitToCelsius(ConvertRequest) returns (ConvertResponse);
  rpc ConvertSolesToDollars(ConvertRequest) returns (ConvertResponse);
  rpc ConvertDollarsToSoles(ConvertRequest) returns (ConvertResponse);
  rpc ConvertKmToMiles(ConvertRequest) returns (ConvertResponse);
  rpc ConvertMilesToKm(ConvertRequest) returns (ConvertResponse);
  rpc ConvertKgToLbs(ConvertRequest) returns (ConvertResponse);
  rpc ConvertLbsToKg(ConvertRequest) returns (ConvertResponse);
  rpc ConvertHoursToMinutes(ConvertRequest) returns (ConvertResponse);
  rpc ConvertMinutesToHours(ConvertRequest) returns (ConvertResponse);
}

message ConvertRequest {
  double input = 1;
}

message ConvertResponse {
  double result = 1;
}
```

Implementación del Servicio

```
package com.lab05.ep2;

import com.lab05.ep2.v1.ConvertRequest;
import com.lab05.ep2.v1.ConvertResponse;
import com.lab05.ep2.v1.ConverterServiceGrpc;
import io.grpc.stub.StreamObserver;

public class ConverterService
    extends ConverterServiceGrpc.ConverterServiceImplBase
{
    @Override
    public void convertCelsiusToFahrenheit(
        ConvertRequest req,
        StreamObserver<ConvertResponse> responseObserver
    ) {
        double result = req.getInput() * 1.8 + 32;
        responseObserver.onNext(
            ConvertResponse.newBuilder().setResult(result).build()
        );
        responseObserver.onCompleted();
    }

    @Override
    public void convertSolesToDollars(
        ConvertRequest req,
        StreamObserver<ConvertResponse> responseObserver
    ) {
        double result = req.getInput() * 0.23;
        responseObserver.onNext(
            ConvertResponse.newBuilder().setResult(result).build()
        );
        responseObserver.onCompleted();
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

```

@Override
public void convertKmToMiles(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 0.621371;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertMilesToKm(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() / 0.621371;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertKgToLbs(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 2.20462;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertLbsToKg(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() / 2.20462;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertHoursToMinutes(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() * 60;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}

@Override
public void convertMinutesToHours(
    ConvertRequest req,
    StreamObserver<ConvertResponse> responseObserver
) {
    double result = req.getInput() / 60;
    responseObserver.onNext(
        ConvertResponse.newBuilder().setResult(result).build()
    );
    responseObserver.onCompleted();
}
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 20

Servidor

```
package com.lab05.ep2;

import io.grpc.Server;
import io.grpc.ServerBuilder;

public class ServerMain {

    public static void main(String[] args) throws Exception {
        Server server = ServerBuilder.forPort(50051)
            .addService(new ConverterService())
            .build();

        server.start();

        System.out.println("Servidor iniciado en puerto 50051");

        server.awaitTermination();
    }
}
```

Cliente CLI

```
package com.lab05.ep2;

import com.lab05.ep2.v1.ConvertRequest;
import com.lab05.ep2.v1.ConvertResponse;
import com.lab05.ep2.v1.ConverterServiceGrpc;
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;

public class Client {

    public static void main(String[] args) {
        ManagedChannel channel = ManagedChannelBuilder.forAddress(
            "localhost",
            50051
        )
            .usePlaintext()
            .build();
        ConverterServiceGrpc.ConverterServiceBlockingStub stub =
            ConverterServiceGrpc.newBlockingStub(channel);

        int opcion = 1;

        while (opcion != 0) {
            System.out.println("Menu de conversión:");
            System.out.println("1. Celsius a Fahrenheit");
            System.out.println("2. Fahrenheit a Celsius");
            System.out.println("3. Soles a Dólares");
            System.out.println("4. Dólares a Soles");
            System.out.println("5. Km a Millas");
            System.out.println("6. Millas a Km");
            System.out.println("7. Kg a Lbs");
            System.out.println("8. Lbs a Kg");
            System.out.println("9. Horas a Minutos");
            System.out.println("10. Minutos a Horas");
            System.out.println("0. Salir");
            System.out.println("Ingrese una opción:");
            opcion = Integer.parseInt(System.console().readLine());
            switch (opcion) {
                case 1:
                    System.out.println("Ingrese el valor a convertir:");
                    double valor = Double.parseDouble(System.console().readLine());
                    ConvertRequest request = ConvertRequest.newBuilder()
                        .setInput(valor)
                        .build();
                    ConvertResponse response = stub.convertCelsiusToFahrenheit(request);
                    System.out.println("Resultado: " + response.getResult());
                    break;
            }
        }
    }
}
```

```
case 2:
    System.out.println("Ingrese el valor a convertir:");
    double valor2 = Double.parseDouble(System.console().readLine());
    ConvertRequest request2 = ConvertRequest.newBuilder()
        .setInput(valor2)
        .build();
    ConvertResponse response2 = stub.convertFahrenheitToCelsius(request2);
    System.out.println("Resultado: " + response2.getResult());
    break;
case 3:
    System.out.println("Ingrese el valor a convertir:");
    double valor3 = Double.parseDouble(System.console().readLine());
    ConvertRequest request3 = ConvertRequest.newBuilder()
        .setInput(valor3)
        .build();
    ConvertResponse response3 = stub.convertSolesToDollars(request3);
    System.out.println("Resultado: " + response3.getResult());
    break;
case 4:
    System.out.println("Ingrese el valor a convertir:");
    double valor4 = Double.parseDouble(System.console().readLine());
    ConvertRequest request4 = ConvertRequest.newBuilder()
        .setInput(valor4)
        .build();
    ConvertResponse response4 = stub.convertDollarsToSoles(request4);
    System.out.println("Resultado: " + response4.getResult());
    break;
case 5:
    System.out.println("Ingrese el valor a convertir:");
    double valor5 = Double.parseDouble(System.console().readLine());
    ConvertRequest request5 = ConvertRequest.newBuilder()
        .setInput(valor5)
        .build();
    ConvertResponse response5 = stub.convertKmToMiles(request5);
    System.out.println("Resultado: " + response5.getResult());
    break;
case 6:
    System.out.println("Ingrese el valor a convertir:");
    double valor6 = Double.parseDouble(System.console().readLine());
    ConvertRequest request6 = ConvertRequest.newBuilder()
        .setInput(valor6)
        .build();
    ConvertResponse response6 = stub.convertMilesToKm(request6);
    System.out.println("Resultado: " + response6.getResult());
    break;
case 7:
    System.out.println("Ingrese el valor a convertir:");
    double valor7 = Double.parseDouble(System.console().readLine());
    ConvertRequest request7 = ConvertRequest.newBuilder()
        .setInput(valor7)
        .build();
    ConvertResponse response7 = stub.convertKgToLbs(request7);
    System.out.println("Resultado: " + response7.getResult());
    break;
case 8:
    System.out.println("Ingrese el valor a convertir:");
    double valor8 = Double.parseDouble(System.console().readLine());
    ConvertRequest request8 = ConvertRequest.newBuilder()
        .setInput(valor8)
        .build();
    ConvertResponse response8 = stub.convertLbsToKg(request8);
    System.out.println("Resultado: " + response8.getResult());
    break;
case 9:
    System.out.println("Ingrese el valor a convertir:");
    double valor9 = Double.parseDouble(System.console().readLine());
    ConvertRequest request9 = ConvertRequest.newBuilder()
        .setInput(valor9)
        .build();
    ConvertResponse response9 = stub.convertHoursToMinutes(request9);
    System.out.println("Resultado: " + response9.getResult());
    break;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 22

```

        case 10:
            System.out.println("Ingrese el valor a convertir:");
            double valor10 = Double.parseDouble(System.console().readLine());
            ConvertRequest request10 = ConvertRequest.newBuilder()
                .setInput(valor10)
                .build();
            ConvertResponse response10 = stub.convertMinutesToHours(request10);
            System.out.println("Resultado: " + response10.getResult());
            break;
        case 0:
            //Salir
            System.out.println("Saliendo...");
            break;
        default:
            System.out.println("Opción inválida");
            break;
    }
}
channel.shutdown();
}
}

```

Cliente GUI

```

package com.lab05.ep2;

import com.lab05.ep2.v1.ConvertRequest;
import com.lab05.ep2.v1.ConvertResponse;
import com.lab05.ep2.v1.ConverterServiceGrpc;
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;
import java.awt.*;
import java.awt.event.*;
import java.text.DecimalFormat;
import java.util.*;
import javax.swing.*;
import javax.swing.border.*;

/**
 * Cliente gRPC con interfaz gráfica Swing.
 * Reemplaza el menú de consola por una UI moderna.
 *
 * Dependencias: igual que el Client.java original (grpc, proto generado).
 * Para compilar junto al proyecto, reemplaza Client.java por este archivo.
 */
public class ClientGUI extends JFrame {

    // — gRPC —————
    private ManagedChannel channel;
    private ConverterServiceGrpc.ConverterServiceBlockingStub stub;
    private boolean connected = false;

    // — Colores —————
    private static final Color BG_DARK = new Color(13, 17, 23);
    private static final Color BG_CARD = new Color(22, 27, 34);
    private static final Color BG_INPUT = new Color(33, 38, 45);
    private static final Color ACCENT = new Color(88, 166, 255);
    private static final Color ACCENT_HOVER = new Color(121, 192, 255);
    private static final Color SUCCESS = new Color(63, 185, 80);
    private static final Color WARNING = new Color(210, 153, 34);
    private static final Color TEXT_PRIMARY = new Color(230, 237, 243);
    private static final Color TEXT_MUTED = new Color(125, 133, 144);
    private static final Color BORDER = new Color(48, 54, 61);

    // — Fuentes —————
    private static final Font FONT_UI = new Font("Segoe UI", Font.PLAIN, 13);
    private static final Font FONT_TITLE = new Font("Segoe UI", Font.BOLD, 15);
    private static final Font FONT_LABEL = new Font("Segoe UI", Font.PLAIN, 11);

    // — Conversiones disponibles —————
    private static final Object[][] CONVERSIONS = {

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 23

```

{"🌡️ Temperatura", null, null},
{"Celsius → Fahrenheit", "°C", "°F"},
{"Fahrenheit → Celsius", "°F", "°C"},
{"💰 Moneda", null, null},
{"Soles → Dólares", "S/.", "$"},
{"Dólares → Soles", "$", "S/."},
{"📏 Distancia", null, null},
{"Km → Millas", "km", "mi"},
{"Millas → Km", "mi", "km"},
{"💵 Peso", null, null},
{"Kg → Libras", "kg", "lbs"},
{"Libras → Kg", "lbs", "kg"},
{"🕒 Tiempo", null, null},
{"Horas → Minutos", "h", "min"},
{"Minutos → Horas", "min", "h"},
};

// --- Widgets ---
private JList<String> conversionList;
private DefaultListModel<String> listModel;
private JTextField inputField;
private JLabel resultLabel;
private JLabel unitInLabel, unitOutLabel;
private JButton convertBtn;
private JLabel statusDot;
private JLabel statusText;
private JTextArea historyArea;
private JLabel selectedLabel;

// mapea índice visual → índice de conversión real (saltando separadores)
private final Map<Integer, Integer> listIndexToConversion =
    new LinkedHashMap<>();

public ClientGUI() {
    setTitle("Convertor gRPC - Lab 05");
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setSize(820, 600);
    setMinimumSize(new Dimension(700, 520));
    setLocationRelativeTo(null);
    setBackground(BG_DARK);

    buildUI();
    connectGRPC();
}

// ---
// Construcción de la UI
// ---
private void buildUI() {
    JPanel root = new JPanel(new BorderLayout()) {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            g.setColor(BG_DARK);
            g.fillRect(0, 0, getWidth(), getHeight());
        }
    };
    root.setOpaque(false);

    root.add(buildTopBar(), BorderLayout.NORTH);
    root.add(buildLeftPanel(), BorderLayout.WEST);
    root.add(buildCenter(), BorderLayout.CENTER);

    setContentPane(root);
}

// --- Barra superior ---
private JPanel buildTopBar() {
    JPanel bar = new JPanel(new BorderLayout());
    bar.setBackground(BG_CARD);
    bar.setBorder(
        new CompoundBorder(

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 24

```

        new MatteBorder(0, 0, 1, 0, BORDER),
        new EmptyBorder(10, 18, 10, 18)
    )
);

JLabel title = new JLabel("  Conversor gRPC");
title.setFont(FONT_TITLE);
title.setForeground(TEXT_PRIMARY);

JPanel statusPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 6, 0));
statusPanel.setOpaque(false);
statusDot = new JLabel("●");
statusDot.setForeground(WARNING);
statusDot.setFont(new Font("Dialog", Font.PLAIN, 10));
statusText = new JLabel("Conectando...");
statusText.setFont(FONT_LABEL);
statusText.setForeground(TEXT_MUTED);

JButton reconnectBtn = smallButton("⚙ Reconectar");
reconnectBtn.addActionListener(e -> connectGRPC());

statusPanel.add(statusDot);
statusPanel.add(statusText);
statusPanel.add(Box.createHorizontalStrut(8));
statusPanel.add(reconnectBtn);

bar.add(title, BorderLayout.WEST);
bar.add(statusPanel, BorderLayout.EAST);
return bar;
}

// — Panel izquierdo: lista de conversiones —————
private JPanel buildLeftPanel() {
    JPanel panel = new JPanel(new BorderLayout());
    panel.setPreferredSize(new Dimension(230, 0));
    panel.setBackground(BG_CARD);
    panel.setBorder(new MatteBorder(0, 0, 0, 1, BORDER));

    JLabel header = new JLabel(" Conversiones");
    header.setFont(new Font("Segoe UI", Font.BOLD, 11));
    header.setForeground(TEXT_MUTED);
    header.setBorder(new EmptyBorder(10, 12, 6, 0));

    listModel = new DefaultListModel<>();
    int convIndex = 0;
    for (int i = 0; i < CONVERSIONS.length; i++) {
        Object[] row = CONVERSIONS[i];
        if (row[1] == null) {
            listModel.addElement("$" + row[0]); // separador
        } else {
            listModel.addElement((String) row[0]);
            listIndexToConversion.put(listModel.size() - 1, convIndex);
            convIndex++;
        }
    }

    conversionList = new JList<>(listModel);
    conversionList.setBackground(BG_CARD);
    conversionList.setForeground(TEXT_PRIMARY);
    conversionList.setFont(FONT_UI);
    conversionList.setFixedCellHeight(34);
    conversionList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    conversionList.setCellRenderer(new ConversionCellRenderer());
    conversionList.setBorder(new EmptyBorder(0, 0, 0, 0));

    // Selección inicial: primera conversión real
    for (int i = 0; i < listModel.size(); i++) {
        if (!listModel.get(i).startsWith("$")) {
            conversionList.setSelectedIndex(i);
            break;
        }
    }
}

```

```
conversionList.addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) onSelectionChange();
});

// impedir selección de separadores
conversionList.addMouseListener(
    new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            int idx = conversionList.locationToIndex(e.getPoint());
            if (
                idx >= 0 && listModel.get(idx).startsWith("$")
            ) conversionList.clearSelection();
        }
    }
);

JScrollPane scroll = new JScrollPane(conversionList);
scroll.setBorder(BorderFactory.createEmptyBorder());
scroll.setBackground(BG_CARD);
scroll.getViewport().setBackground(BG_CARD);
scroll.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);

panel.add(header, BorderLayout.NORTH);
panel.add(scroll, BorderLayout.CENTER);
return panel;
}

// — Panel central: entrada, resultado, historial —
private JPanel buildCenter() {
    JPanel center = new JPanel(new BorderLayout(0, 0));
    center.setBackground(BG_DARK);

    // — Panel de conversión —
    JPanel convertPanel = new JPanel();
    convertPanel.setLayout(new BoxLayout(convertPanel, BoxLayout.Y_AXIS));
    convertPanel.setBackground(BG_DARK);
    convertPanel.setBorder(new EmptyBorder(28, 28, 20, 28));

    selectedLabel = new JLabel("Celsius → Fahrenheit");
    selectedLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
    selectedLabel.setForeground(TEXT_PRIMARY);
    selectedLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

    JLabel subLabel = new JLabel("Ingresa el valor y presiona Convertir");
    subLabel.setFont(FONT_LABEL);
    subLabel.setForeground(TEXT_MUTED);
    subLabel.setAlignmentX(Component.LEFT_ALIGNMENT);
    subLabel.setBorder(new EmptyBorder(4, 0, 20, 0));

    // — Fila de entrada —
    JPanel inputRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
    inputRow.setOpaque(false);
    inputRow.setAlignmentX(Component.LEFT_ALIGNMENT);
    inputRow.setMaximumSize(new Dimension(Integer.MAX_VALUE, 52));

    unitInLabel = new JLabel("°C");
    unitInLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));
    unitInLabel.setForeground(ACCENT);
    unitInLabel.setPreferredSize(new Dimension(44, 44));
    unitInLabel.setHorizontalAlignment(SwingConstants.CENTER);
    unitInLabel.setBackground(BG_INPUT);
    unitInLabel.setOpaque(true);
    unitInLabel.setBorder(new MatteBorder(1, 1, 1, 0, BORDER));

    inputField = new JTextField("0");
    inputField.setPreferredSize(new Dimension(180, 44));
    inputField.setFont(new Font("JetBrains Mono", Font.PLAIN, 16));
    inputField.setBackground(BG_INPUT);
    inputField.setForeground(TEXT_PRIMARY);
    inputField.setCaretColor(ACCENT);
}
```

```
inputField.setBorder(new MatteBorder(1, 0, 1, 0, BORDER));
inputField.setHorizontalAlignment(JTextField.RIGHT);
inputField.addActionListener(e -> doConvert());

JLabel arrow = new JLabel(" → ");
arrow.setFont(new Font("Segoe UI", Font.PLAIN, 18));
arrow.setForeground(TEXT_MUTED);

inputRow.add(unitInLabel);
inputRow.add(inputField);
inputRow.add(arrow);

// — Resultado —
JPanel resultBox = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
resultBox.setOpaque(false);
resultBox.setAlignmentX(Component.LEFT_ALIGNMENT);
resultBox.setMaximumSize(new Dimension(Integer.MAX_VALUE, 52));

resultLabel = new JLabel("-");
resultLabel.setFont(new Font("JetBrains Mono", Font.BOLD, 20));
resultLabel.setForeground(SUCCESS);
resultLabel.setPreferredSize(new Dimension(180, 44));
resultLabel.setHorizontalAlignment(SwingConstants.RIGHT);
resultLabel.setBackground(BG_INPUT);
resultLabel.setOpaque(true);
resultLabel.setBorder(
    new CompoundBorder(
        new MatteBorder(1, 0, 1, 0, BORDER),
        new EmptyBorder(0, 8, 0, 8)
    )
);

unitOutLabel = new JLabel("°F");
unitOutLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));
unitOutLabel.setForeground(SUCCESS);
unitOutLabel.setPreferredSize(new Dimension(44, 44));
unitOutLabel.setHorizontalAlignment(SwingConstants.CENTER);
unitOutLabel.setBackground(BG_INPUT);
unitOutLabel.setOpaque(true);
unitOutLabel.setBorder(new MatteBorder(1, 0, 1, 1, BORDER));

resultBox.add(resultLabel);
resultBox.add(unitOutLabel);

// — Botón convertir —
convertBtn = new JButton("Convertir");
convertBtn.setFont(new Font("Segoe UI", Font.BOLD, 13));
convertBtn.setBackground(ACCENT);
convertBtn.setForeground(BG_DARK);
convertBtn.setBorder(new EmptyBorder(10, 24, 10, 24));
convertBtn.setFocusPainted(false);
convertBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
convertBtn.setOpaque(true);
convertBtn.setAlignmentX(Component.LEFT_ALIGNMENT);
convertBtn.addActionListener(e -> doConvert());
convertBtn.addMouseListener(
    new MouseAdapter() {
        @Override
        public void mouseEntered(MouseEvent e) {
            convertBtn.setBackground(ACCENT_HOVER);
        }

        @Override
        public void mouseExited(MouseEvent e) {
            convertBtn.setBackground(ACCENT);
        }
    }
);

JButton clearBtn = smallButton("Limpiar historial");
clearBtn.setAlignmentX(Component.LEFT_ALIGNMENT);
clearBtn.addActionListener(e -> historyArea.setText(""));
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 27

```

JPanel btnRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 0));
btnRow.setOpaque(false);
btnRow.setAlignmentX(Component.LEFT_ALIGNMENT);
btnRow.add(convertBtn);
btnRow.add(clearBtn);

convertPanel.add(selectedLabel);
convertPanel.add(subLabel);
convertPanel.add(inputRow);
convertPanel.add(Box.createVerticalStrut(12));
convertPanel.add(resultBox);
convertPanel.add(Box.createVerticalStrut(20));
convertPanel.add(btnRow);

// — Historial —————
JPanel histPanel = new JPanel(new BorderLayout());
histPanel.setBackground(BG_DARK);
histPanel.setBorder(
    new CompoundBorder(
        new MatteBorder(1, 0, 0, 0, BORDER),
        new EmptyBorder(0, 28, 0, 28)
    )
);

JLabel histHeader = new JLabel("Historial de conversiones");
histHeader.setFont(new Font("Segoe UI", Font.BOLD, 11));
histHeader.setForeground(TEXT_MUTED);
histHeader.setBorder(new EmptyBorder(10, 0, 6, 0));

historyArea = new JTextArea();
historyArea.setEditable(false);
historyArea.setBackground(BG_CARD);
historyArea.setForeground(TEXT_PRIMARY);
historyArea.setFont(new Font("JetBrains Mono", Font.PLAIN, 12));
historyArea.setBorder(new EmptyBorder(8, 10, 8, 10));
historyArea.setLineWrap(false);

JScrollPane histScroll = new JScrollPane(historyArea);
histScroll.setBorder(new LineBorder(BORDER, 1));
histScroll.setBackground(BG_CARD);
histScroll.getViewport().setBackground(BG_CARD);
histScroll.setPreferredSize(new Dimension(0, 140));

histPanel.add(histHeader, BorderLayout.NORTH);
histPanel.add(histScroll, BorderLayout.CENTER);
histPanel.add(Box.createVerticalStrut(16), BorderLayout.SOUTH);

center.add(convertPanel, BorderLayout.CENTER);
center.add(histPanel, BorderLayout.SOUTH);

return center;
}

// —————
// Lógica
// —————

private void connectGRPC() {
    statusDot.setForeground(WARNING);
    statusText.setText("Conectando...");
    new SwingWorker<Boolean, Void>() {
        @Override
        protected Boolean doInBackground() {
            try {
                if (channel != null && !channel.isShutdown()) channel.shutdown();
                channel = ManagedChannelBuilder.forAddress("localhost", 50051)
                    .usePlaintext()
                    .build();
                stub = ConverterServiceGrpc.newBlockingStub(channel);
                // Prueba rápida
                ConvertRequest req = ConvertRequest.newBuilder().setInput(0).build();
            }
        }
    }.execute();
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 28

```

        stub.convertCelsiusToFahrenheit(req);
        return true;
    } catch (Exception e) {
        return false;
    }
}

@Override
protected void done() {
    try {
        connected = get();
    } catch (Exception ex) {
        connected = false;
    }
    if (connected) {
        statusDot.setForeground(SUCCESS);
        statusText.setText("Conectado a localhost:50051");
        appendHistory("✓ Conectado al servidor gRPC (localhost:50051)");
    } else {
        statusDot.setForeground(new Color(248, 81, 73));
        statusText.setText("Sin conexión");
        appendHistory(
            "x No se pudo conectar. Verifica que el servidor esté activo."
        );
    }
}
}.execute();
}

private void onSelectionChange() {
    int idx = conversionList.getSelectedIndex();
    if (idx < 0 || listModel.get(idx).startsWith("$")) return;

    Object[] row = getConversionRow(idx);
    selectedLabel.setText((String) row[0]);
    unitInLabel.setText((String) row[1]);
    unitOutLabel.setText((String) row[2]);
    resultLabel.setText("-");
    inputField.selectAll();
    inputField.requestFocus();
}

private void doConvert() {
    if (!connected) {
        JOptionPane.showMessageDialog(
            this,
            "No hay conexión con el servidor.\nPresiona ◀ Reconectar.",
            "Sin conexión",
            JOptionPane.WARNING_MESSAGE
        );
        return;
    }
    int listIdx = conversionList.getSelectedIndex();
    if (listIdx < 0 || listModel.get(listIdx).startsWith("$")) {
        JOptionPane.showMessageDialog(
            this,
            "Selecciona una conversión.",
            "Aviso",
            JOptionPane.INFORMATION_MESSAGE
        );
        return;
    }

    double input;
    try {
        input = Double.parseDouble(inputField.getText().trim().replace(",", "."));
    } catch (NumberFormatException ex) {
        resultLabel.setForeground(new Color(248, 81, 73));
        resultLabel.setText("Error");
        return;
    }
}

```

```
ConvertRequest req = ConvertRequest.newBuilder().setInput(input).build();
Integer convIdx = listIndexToConversion.get(listIdx);
if (convIdx == null) return;

Object[] row = getConversionRow(listIdx);
String name = (String) row[0];
String unitIn = (String) row[1];
String unitOut = (String) row[2];

new SwingWorker<GRPCResult, Void>() {
    @Override
    protected GRPCResult doInBackground() throws Exception {
        return callGRPC(convIdx, req);
    }

    @Override
    protected void done() {
        try {
            GRPCResult res = get();
            DecimalFormat df = new DecimalFormat("#,##0.####");
            resultLabel.setForeground(SUCCESS);
            resultLabel.setText(df.format(res.value()));
            String line = String.format(
                "%-26s %10s %-4s → %10s %s (%d mms)",
                name,
                df.format(input),
                unitIn,
                df.format(res.value()),
                unitOut,
                res.ms()
            );
            appendHistory(line);
        } catch (Exception ex) {
            resultLabel.setForeground(new Color(248, 81, 73));
            resultLabel.setText("Error RPC");
            appendHistory("x Error al llamar al servidor: " + ex.getCause());
        }
    }
}.execute();
}

/** Llama al método GRPC correcto según el índice de conversión real. */
private GRPCResult callGRPC(int convIdx, ConvertRequest req) {
    // convIdx: 0-9 saltando los separadores
    // Orden: CelToF, FToCel, SolesToDol, DolToSol, KmToMi, MiToKm, KgToLbs, LbsToKg, HrsToMin, MinToHrs
    ConvertResponse resp;
    long start = System.nanoTime();
    switch (convIdx) {
        case 0:
            resp = stub.convertCelsiusToFahrenheit(req);
            break;
        case 1:
            resp = stub.convertFahrenheitToCelsius(req);
            break;
        case 2:
            resp = stub.convertSolesToDollars(req);
            break;
        case 3:
            resp = stub.convertDollarsToSoles(req);
            break;
        case 4:
            resp = stub.convertKmToMiles(req);
            break;
        case 5:
            resp = stub.convertMilesToKm(req);
            break;
        case 6:
            resp = stub.convertKgToLbs(req);
            break;
        case 7:
            resp = stub.convertLbsToKg(req);
            break;
    }
}
```



```
        case 8:
            resp = stub.convertHoursToMinutes(req);
            break;
        case 9:
            resp = stub.convertMinutesToHours(req);
            break;
        default:
            throw new IllegalStateException("Índice inválido: " + convIdx);
    }
    long ms = (System.nanoTime() - start) / 1_000;
    return new GRPCResult(resp.getResult(), ms);
}

private Object[] getConversionRow(int listIdx) {
    // Cuenta cuántos separadores hay antes de listIdx para mapear al CONVERSIONS correcto
    int skip = 0;
    for (int i = 0; i <= listIdx; i++) {
        if (listModel.get(i).startsWith("$")) skip++;
    }
    return CONVERSIONS[listIdx]; // el listModel refleja CONVERSIONS 1:1
}

private void appendHistory(String text) {
    SwingUtilities.invokeLater(() -> {
        historyArea.append(text + "\n");
        historyArea.setCaretPosition(historyArea.getDocument().getLength());
    });
}

// -----
// Helpers de UI
// -----

private JButton smallButton(String text) {
    JButton btn = new JButton(text);
    btn.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    btn.setBackground(BG_INPUT);
    btn.setForeground(TEXT_MUTED);
    btn.setBorder(
        new CompoundBorder(
            new LineBorder(BORDER, 1),
            new EmptyBorder(6, 12, 6, 12)
        )
    );
    btn.setFocusPainted(false);
    btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btn.setOpaque(true);
    btn.addMouseListener(
        new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                btn.setForeground(TEXT_PRIMARY);
            }

            @Override
            public void mouseExited(MouseEvent e) {
                btn.setForeground(TEXT_MUTED);
            }
        }
    );
    return btn;
}

// -----
// Cell renderer personalizado para la lista
// -----
class ConversionCellRenderer extends DefaultListCellRenderer {

    @Override
    public Component getListCellRendererComponent(
        JList<?> list,
        Object value,
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 31

```

        int index,
        boolean isSelected,
        boolean cellHasFocus
    ) {
        String text = (String) value;
        boolean isSep = text.startsWith("$");

        JLabel label = (JLabel) super.getListCellRendererComponent(
            list,
            isSep ? text.substring(1) : text,
            index,
            isSelected && !isSep,
            cellHasFocus && !isSep
        );

        label.setBackground(BG_CARD);
        label.setOpaque(true);

        if (isSep) {
            label.setFont(new Font("Segoe UI", Font.BOLD, 10));
            label.setForeground(TEXT_MUTED);
            label.setBorder(
                new CompoundBorder(
                    new MatteBorder(isSepFirst(index) ? 0 : 1, 0, 0, 0, BORDER),
                    new EmptyBorder(8, 12, 2, 0)
                )
            );
        } else if (isSelected) {
            label.setBackground(new Color(33, 52, 78));
            label.setForeground(ACCENT);
            label.setBorder(
                new CompoundBorder(
                    new MatteBorder(0, 2, 0, 0, ACCENT),
                    new EmptyBorder(0, 14, 0, 0)
                )
            );
        } else {
            label.setForeground(TEXT_PRIMARY);
            label.setBorder(new EmptyBorder(0, 16, 0, 0));
        }
        return label;
    }

    private boolean isSepFirst(int idx) {
        return idx == 0;
    }
}

// -----
// Main
// -----
public static void main(String[] args) {
    // Apariencia nativa del sistema
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception ignored) {}

    // Forzar tema oscuro en scroll bars
    UIManager.put("ScrollBar.background", BG_CARD);
    UIManager.put("ScrollBar.thumb", BORDER);
    UIManager.put("ScrollBar.track", BG_CARD);
    UIManager.put("ScrollBar.thumbHighlight", TEXT_MUTED);
    UIManager.put("ScrollBar.thumbDarkShadow", BORDER);
    UIManager.put("ScrollBar.thumbShadow", BORDER);

    SwingUtilities.invokeLater(() -> new ClientGUI().setVisible(true));
}

private record GRPCResult(double value, long ms) {}
}

```